m4rk3t

Campus Marketplace

Software Design Document

Architectural Design

Course: Senior Project I

Date: November 3, 2026

Names: Marleena Limbrick, Vicente Anyaegbu, Temi Iredele, Ravonne Clemons, Jordan Chase

6.1 Introduction

This document presents the architecture and detailed design for the software for the **m4rk3t (Campus Marketplace)** project. m4rk3t is a web application that lets college students find and offer services and goods inside their own campus community. Students register with a school (.edu) email address as either a *customer* or a *vendor*. Customers search and filter listings by category and campus, open listing details, and view public business profiles. Vendors maintain a storefront profile and use a vendor studio to create, edit, hide, and delete their listings. The software consists of a responsive single-page frontend and a Python REST backend that stores its data in SQLite by default, with PostgreSQL supported through configuration.

The current release is a development prototype. Bookings, payments, messaging, reviews, notifications, and moderation are not part of this design. Where the interface shows them, it shows honest empty or unavailable states instead of invented data. Section 6.2 gives the architecture and eventually section 6.3 will give the detailed design of every component, class, interface, and data structure, with the supporting UML diagrams.

6.1.1 System Objectives

The objectives of m4rk3t go beyond listing items for sale. The application is designed to:

- **Keep commerce inside the campus community.** Campus and category filters let a student find offerings from peers at their own school instead of searching a general marketplace. • **Make selling easy for students.** A student can register as a vendor, fill in a storefront profile, and publish a first listing in a few minutes, and can hide or remove it just as quickly. • **Build trust through honest behavior.** Registration requires a .edu address format and a minimum password length; only a listing's owner can change it; and the interface never fabricates bookings, reviews, followers, or vendors. Example listings are clearly labeled and cannot be edited. • **Be a base that can grow.** A layered design with a single API gateway on the frontend and a single security module on the backend means that bookings, reviews, and email verification can be added later without restructuring the existing code.
- **Be easy to run and to move.** The project runs locally with two commands, and the database can be switched from SQLite to PostgreSQL by changing one environment variable.

6.1.2 Hardware, Software, and Human Interfaces

Each external interface is numbered below. Versions written as "or later" are the minimums needed to run the project. The exact versions of third-party packages are pinned in backend/requirements.txt and frontend/package.json.

Hardware interfaces

6.1.2.1 Client display, keyboard, pointing device, and touch

Students use a laptop, desktop, tablet, or phone. The layout is responsive from phone to desktop widths. Input is by keyboard (forms, search box) and by mouse or touch (buttons, dialogs, filter menus). No camera, microphone, or location sensor is used.

6.1.2.2 Network interface

Client and server communicate over TCP/IP (Wi-Fi or Ethernet) using HTTP/1.1 with JSON bodies. In development, the browser loads the frontend from http://localhost:5173 and calls the backend at http://localhost:8000. Any real deployment must use HTTPS, and the backend's CORS allowlist must be set to the deployed frontend origin. The browser also needs internet access to load the example listing images (6.1.2.11).

6.1.2.3 Server hardware and storage

The backend runs on any machine that can run Python 3.10 or later (a developer workstation in this prototype). With the default SQLite configuration, the database is a single file on the server's local disk and needs no separate database host. A PostgreSQL deployment uses a separate database host reached over TCP.

Software interfaces

6.1.2.4 Web browser

A current version of Chrome, Edge, Firefox, or Safari with JavaScript enabled. The frontend uses the Fetch API for HTTP and the Web Storage API (localStorage and sessionStorage) to keep the login session. Interfaces with: ApiClient, SessionStore.

6.1.2.5 Python runtime and Uvicorn

Python 3.10 or later runs the backend. Uvicorn, an ASGI server, serves the application on port 8000 with uvicorn main:app --reload --port 8000. Interfaces with: AppMain.

6.1.2.6 FastAPI

The web framework for routing, dependency injection (get_db, get_current_user), request and response validation with Pydantic models, and automatic OpenAPI documentation at /docs. Version: as pinned in requirements.txt. Interfaces with: AppMain, AuthRouter, ProfileRouter, ListingRouter, AuthDependencies.

6.1.2.7 SQLAlchemy and database drivers

SQLAlchemy maps the User, VendorProfile, and Listing classes to database tables. SQLite through Python's built-in sqlite3 is the default. PostgreSQL is selected by setting DATABASE_URL and installing a PostgreSQL driver. Version: as pinned in requirements.txt. Interfaces with: Database, User, VendorProfile, Listing, and the routers.

6.1.2.8 JWT and password hashing libraries

A JSON Web Token library signs and verifies access tokens (HS256, keyed by SECRET_KEY), and a password hashing library produces salted password hashes. Both are used only inside SecurityService, so the rest of the code does not depend on them. Version: as pinned in requirements.txt.

6.1.2.9 Node.js, npm, and Vite

Node.js 20 or later with npm installs the frontend dependencies. The Vite development server serves the frontend on port 5173 (npm run dev) and builds the production bundle.

6.1.2.10 Frontend UI library

The Presentation and Client Services components are a single-page application built with a component-based JavaScript UI library (React). Navigation is by switching views inside AppShell, not by separate URL routes. Version: as pinned in package.json.

6.1.2.11 Unsplash image hosting (external service)

The six example listings reference photographs hosted on Unsplash. The browser fetches them directly with HTTPS GET; the backend is not involved. When the browser is offline, the images do not load and the card still shows its text.

6.1.2.12 Environment configuration

DATABASE_URL selects the database (default: a local SQLite file). SECRET_KEY is the JWT signing key and must be a long random value in any deployment. The frontend's API base URL defaults to http://localhost:8000.

Human interfaces

6.1.2.13 Visual design

The interface follows the supplied wireframes. All text uses Georgia serif typography, with the following palette:

Color	Hex	Used for
Dark red	#780000	Headings, 'Example' badges, price text
Red	#C1121F	Primary buttons and accents
Cream	#FDF0D5	Page background and card accents

Navy	#003049	Top navigation bar, body text, outlines
Light blue	#669BBC	Image placeholders and secondary accents

6.1.2.14 Screens

The interface is a single-page app. The implemented screens are: landing page; login and sign-up dialogs; Discover (search, category and campus filters, listing cards); listing (service) details; public business profile; My m4rk3t (account summary, appointments empty state, unavailable follower and review counts); profile editing; and the vendor studio (listing creation and editing). Figure 1 and Figure 2 are simulated drawings of the Discover screen and the vendor studio.

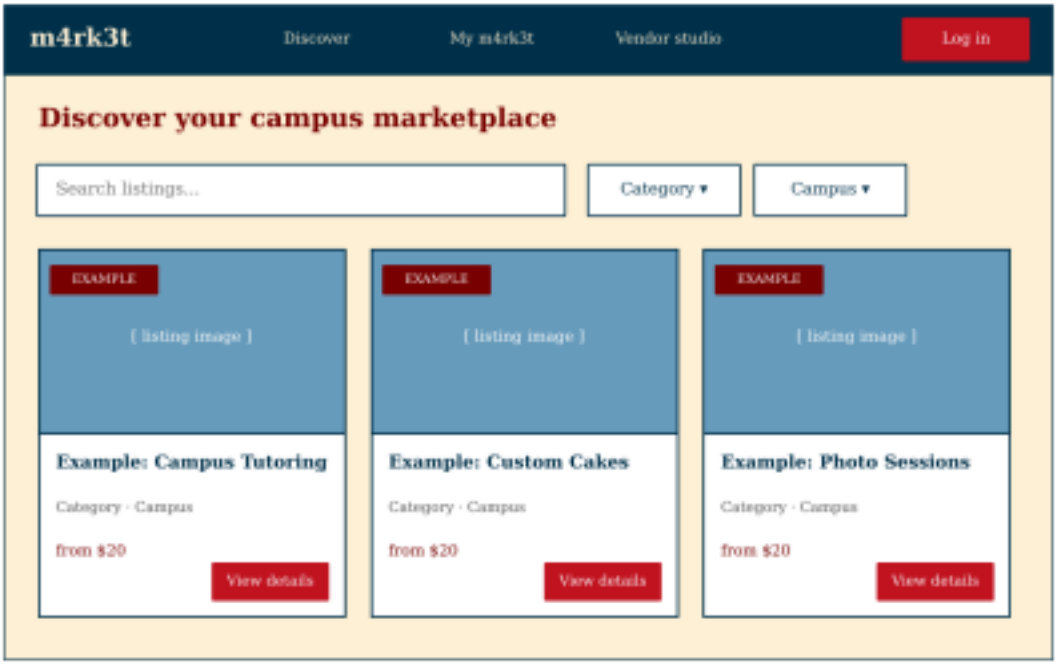


Figure 1. *Simulated drawing of the Discover screen (search, filters, and labeled example listings).*

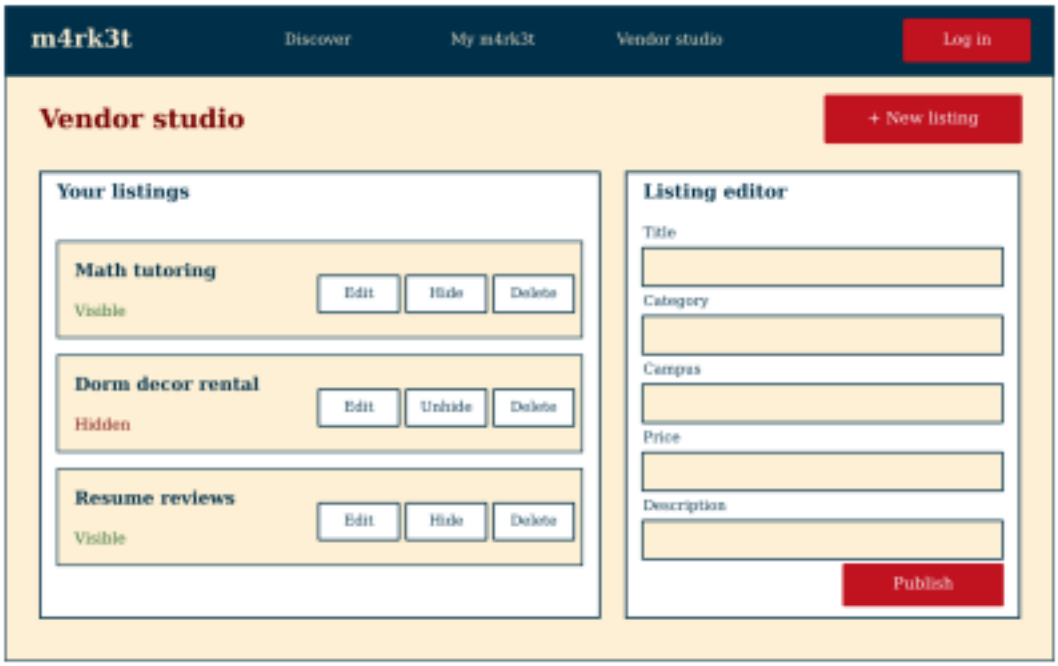


Figure 2. *Simulated drawing of the vendor studio (listing list with Edit, Hide/Unhide, Delete, and the listing editor).*

6.1.2.15 Interaction behavior

Forms show inline validation messages (for example, a non-.edu email or a password shorter than 8 characters). The login dialog has a Remember me option that keeps the session across browser restarts. Deleting a listing asks for confirmation. Hidden listings are marked as hidden in the vendor studio and do not appear in Discover. Screens for features that do not exist yet show an empty or "Unavailable" state rather than made-up numbers.

6.2 Architectural Design

m4rk3t uses a three-layer client-server architecture: a **presentation layer** (the single-page app), an **application layer** (a FastAPI REST API with a security module), and a **data layer** (SQLAlchemy models over SQLite or PostgreSQL). Dependencies point only downward: screens call client services, client services call the REST API, the API calls the security and data components, and the data components call the database. Nothing calls upward, so each layer can be changed or tested on its own. The application is partitioned into five Computer Software Components (CSCs) that together form the **m4rk3t CSCI**: Presentation, Client Services, API, Security, and Data. Section 6.3 will list the classes (CSUs) in each.

Design decisions made by looking for awkwardness

- **One door to the backend.** Every screen reaches the server through ApiClient. Without this, each screen would build its own requests and handle tokens and errors differently (high coupling). With it, the base URL, bearer header, and error handling are written once.
- **The server is authoritative.** The .edu check and the password-length rule exist in both AuthDialog and AuthRouter. The frontend copy only gives fast feedback; the server repeats every check, because anyone can call the API without the UI.
- **Ownership is checked in one place.** Edit, hide, and delete all need "is this my listing?". A single helper in ListingRouter answers it, so a future route cannot forget the check.
- **Account data and storefront data are separate.** VendorProfile is separate from User, so the public profile endpoint never touches the email or password hash, and customers carry no empty storefront fields.
- **API payloads are not database rows.** Request and response schemas are separate from ORM models, so hashed_password can never be returned by accident, and the database can change without changing the API.
- **Examples live only in the frontend.** The six example listings are static labeled data, not rows in the database, so no fake vendors can ever be edited, searched as real, or mistaken for real accounts.

6.2.1 Major Software Components

The table maps each functional requirement to the components that implement it. Requirement identifiers FR-1 to FR-8 follow the order of the project's feature list in the requirements document (section 5.2).

ID	Functional requirement	Presentation / Client Services	Backend (API / Security / Data)
FR-1	Storefront: search, category and campus filters, listing details, responsive layout	DiscoverView, ListingDetailView, ApiClient	ListingRouter, Listing
FR-2	Student registration with .edu format check and optional graduation year	AuthDialog, AuthProvider	AuthRouter, SecurityService, User, VendorProfile
FR-3	Login, JWT authentication, customer/vendor roles, Remember me	AuthDialog, AuthProvider, SessionStore, AppShell (guard)	AuthRouter, SecurityService, AuthDependencies
FR-4	Profile editing (name, bio; vendors: business name)	ProfileEditor, MyMarketView	ProfileRouter, User, VendorProfile
FR-5	Vendor storefront profile and public business profile	BusinessProfileView, ProfileEditor	ProfileRouter, VendorProfile
FR-6	Vendor dashboard: create, edit, hide,	VendorStudio, ApiClient	ListingRouter,

	delete listings		AuthDependencies, Listing

ID	Functional requirement	Presentation / Client Services	Backend (PI / Security / Data)
FR-7	Persistence in SQLite or PostgreSQL; additive startup migration	n/a	Database, AppMain, all models
FR-8	Labeled examples and honest empty/unavailable states	DiscoverView (mergeExamples), MyMarketView, BusinessProfileView	n/a (no fabricated records stored)

Presentation CSC contains the screens and dialogs. **Client Services CSC** holds the login state (AuthProvider), the storage of the session (SessionStore), and the HTTP gateway (ApiClient). **API CSC** exposes the REST endpoints and starts the application. **Security CSC** hashes passwords, issues and checks tokens, and enforces roles. **Data CSC** defines the stored objects and the database connection.

6.2.2 Major Software Interactions

The components communicate through the mechanisms below.

#	From to	Mechanism	What is exchanged
1	Views to AuthProvider and ApiClient	In-process function calls and the UI library's context mechanism	User state, form values, filters, navigation requests
2	AuthProvider and ApiClient to SessionStore	Web Storage API (localStorage or sessionStorage)	One JSON session record: token and user
3	ApiClient to the API CSC	HTTP/1.1 with JSON through the Fetch API; Authorization: Bearer header; CORS	Requests and responses of the REST API
4	Routers to the Security CSC	FastAPI dependency injection and direct function calls	Tokens, decoded claims, password hashes, verdicts
5	Routers to the Data CSC	SQLAlchemy Session (one per request)	ORM objects: User, VendorProfile, Listing
6	Data CSC to the database	SQL through the DB-API driver (sqlite3 or a PostgreSQL driver)	SQL statements and result rows
7	Browser to Unsplash	HTTPS GET of image files	Image bytes for example listings
8	AppMain to Database at startup	Function calls	Table creation and the additive graduation_year column migration

6.2.3 Architectural Design Diagrams

Four architectural UML diagrams follow: a use case diagram (Figure 3), a top-level class diagram (Figure 4), a component and deployment diagram (Figure 5), and a swim lane diagram (Figure 6) that follows a vendor from registration to publishing a first listing.

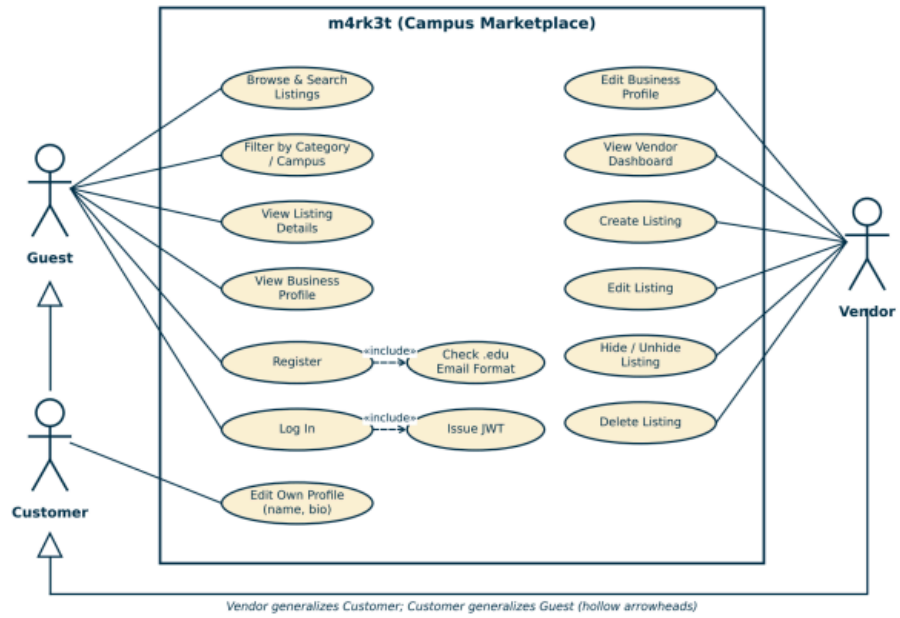


Figure 3. *Use case diagram of m4rk3t. Guests browse and register; customers edit their profile; vendors manage a storefront and listings.*

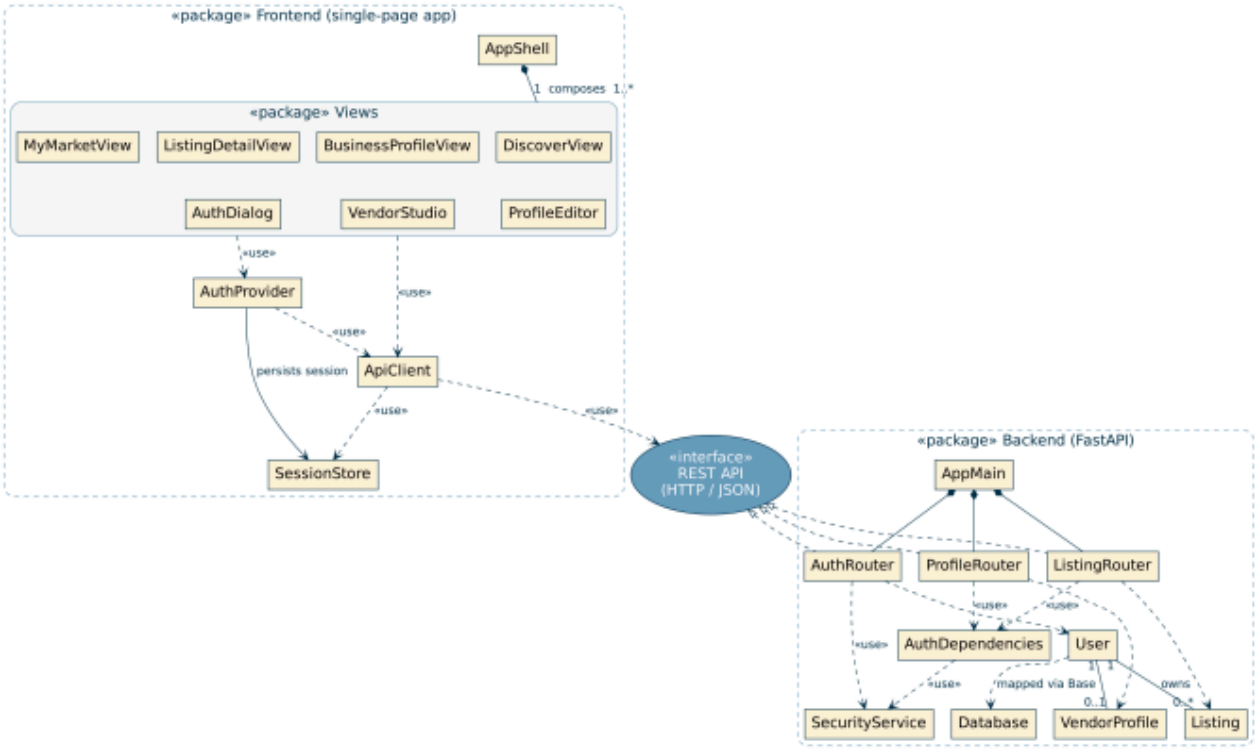


Figure 4. *Top-level class diagram showing the major classes, their packages, and the REST API interface that separates frontend from backend.*

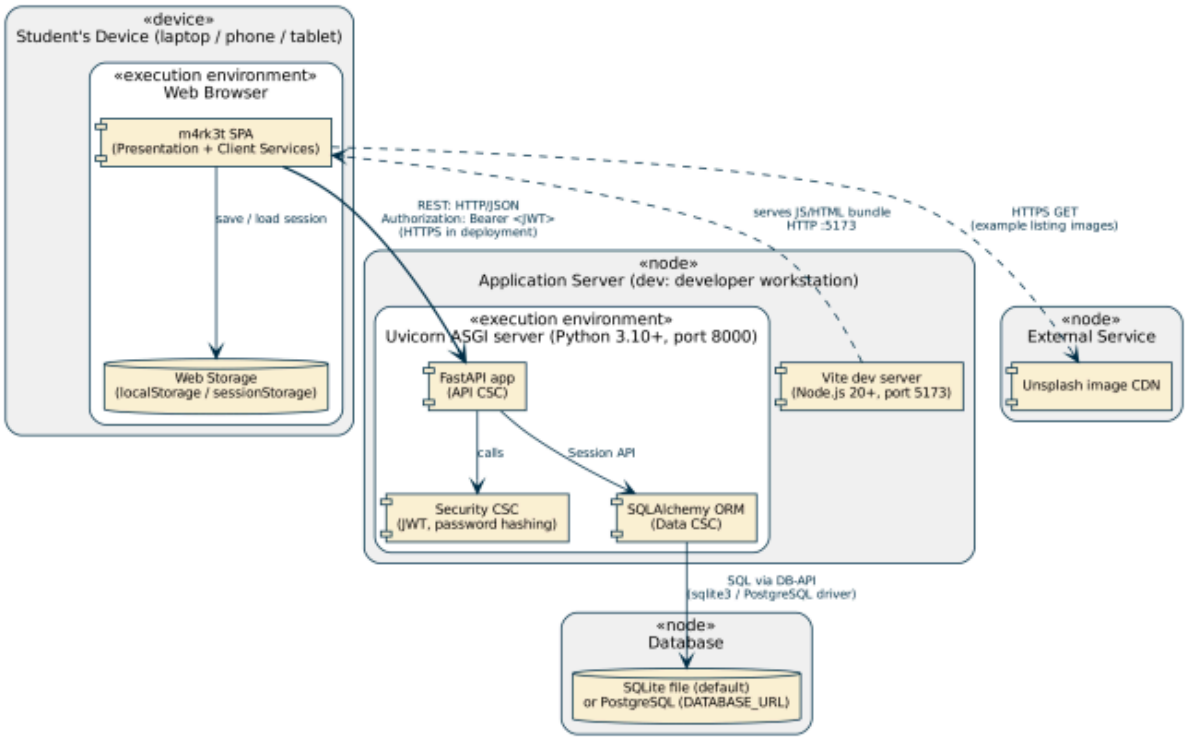


Figure 5. *Component and deployment diagram showing the browser, application server, database, and the external image service.*

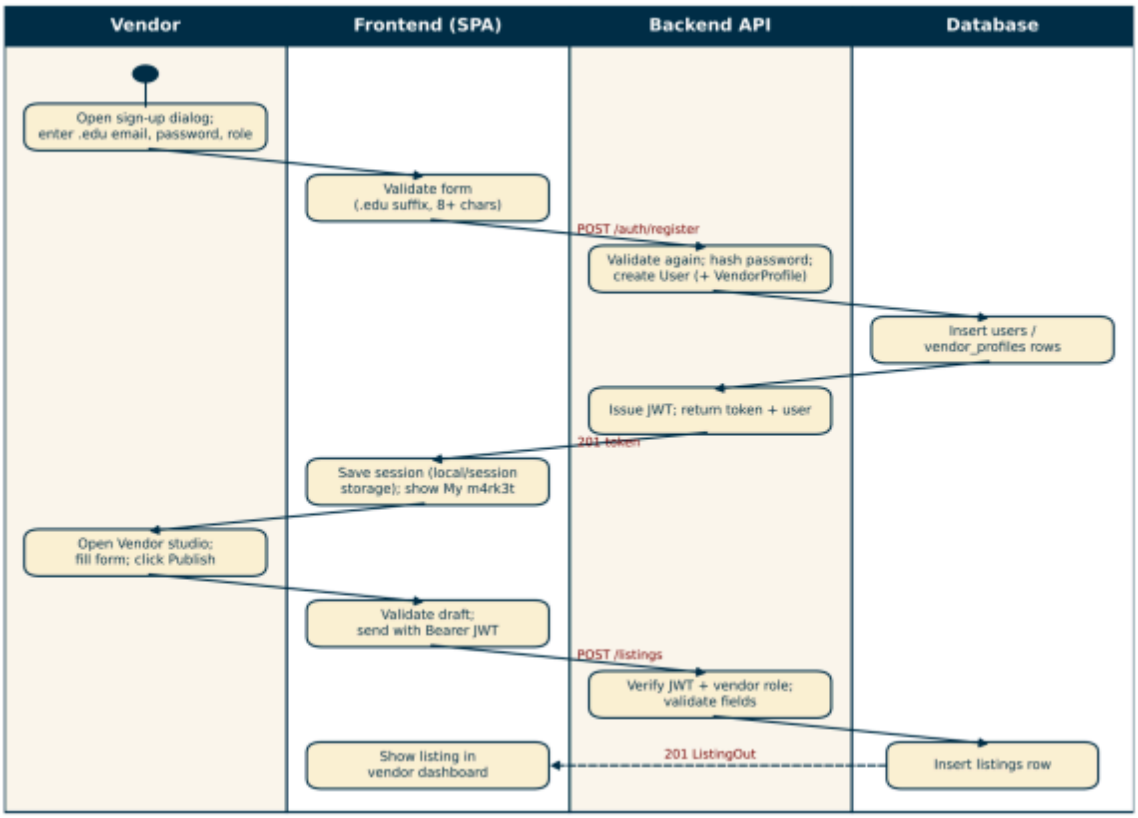


Figure 6. *Swim lane diagram: a vendor registers and publishes a listing across the vendor, frontend, backend API, and database.*